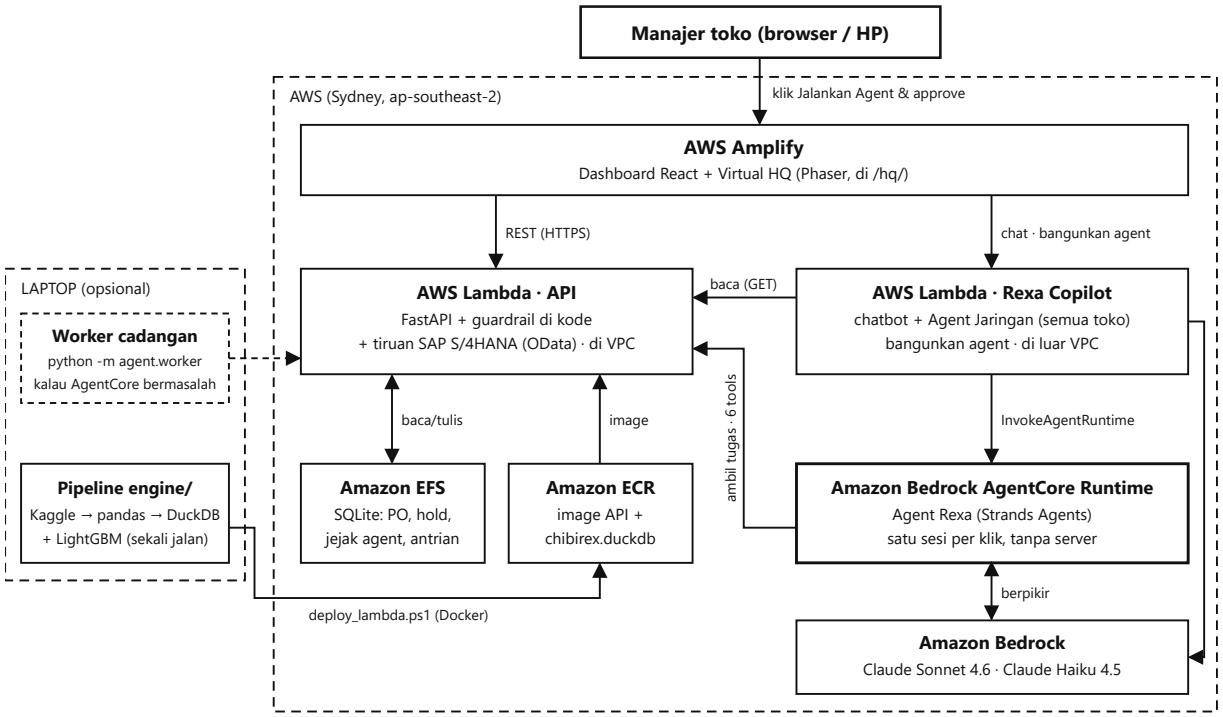
Komponen Teknologi Rexa

Smart Inventory & Cash Flow Copilot · Tim ChibiRex · Agentic AI Hackathon 2026 · kondisi per 26 Oktober 2026

Ringkasnya: agent AI hanya **memilih dan menjelaskan** tindakan; **angka dihitung oleh data** (DuckDB dan model LightGBM), **aturan dijaga oleh kode** (guardrail di API), dan **keputusan akhir tetap di tangan manusia** (1-Click Approval sebelum PO masuk SAP). **Semua bagian online berjalan di AWS, termasuk agent**: laptop tidak perlu menyala saat demo.

Diagram arsitektur



Dashboard, Virtual HQ, API, guardrail, penyimpanan, tiruan SAP, dan agent berjalan di AWS. Laptop hanya dipakai untuk membangun data dan deploy; worker di laptop hanya cadangan kalau AgentCore bermasalah.

1. Data dan analitik ("bahan" dan "hitungan")

Komponen	Untuk apa	Berjalan di
Kaggle	Sumber dataset. **Bibitor**: stok, penjualan, dan pembelian 80 toko minuman. **Instacart**: isi keranjang belanja, untuk cek substitusi/peran komplementer. **Product Backorders**: melatih model risiko stok habis. **Online Retail II**: pola pesanan borongan untuk menguji guardrail.	Diunduh sekali di laptop
Python + pandas	Bahasa utama proyek dan alat olah data.	Laptop
DuckDB	Database analitik dalam **satu file** (`chibirex.duckdb`, ±300 MB). Menghitung kecepatan jual (Vs), reorder point, safety stock, aturan substitusi, Health Score, dan menyimpan skor risiko. Cepat untuk data besar dan tidak perlu server database.	Dibuat di laptop; ikut di image Lambda (hanya dibaca)
LightGBM	Model machine learning yang memperkirakan SKU berisiko kehabisan stok; sumber **Confidence Score** 0–100 (TINGGI ≥ 90, SEDANG ≥ 70). Di data uji, ±2,9× lebih tepat daripada aturan sederhana.	Dilatih di laptop
Pipeline engine/	Rangkaian langkah `python -m engine.pipeline` yang mengubah data mentah Kaggle menjadi `chibirex.duckdb`.	Laptop

2. API dan guardrail ("aturan main")

Komponen	Untuk apa	Berjalan di
FastAPI	Membuat **REST API**: satu-satunya pintu untuk semua aksi. Berisi 6 tools agent dan endpoint dashboard (Health Score, statistik 80 toko, antrian agent, rekap agent per hari, approval PO).	AWS Lambda (online) atau laptop
Pydantic	Memeriksa setiap data yang masuk; field yang tidak dikenal ditolak (agent tidak bisa menyelipkan `"status": "APPROVED"`).	Di dalam API
Guardrail di kode	Pengaman yang tidak bisa dilewati agent atau prompt (rincian di tabel berikut).	Di dalam API
Jam simulasi	Memajukan atau memundurkan waktu dari 31 Des 2016 (maks. 90 hari): stok berkurang sesuai permintaan harian dari data, PO yang disetujui tiba setelah lead time vendor, sehingga agent bisa diuji bereaksi ke kondisi baru.	Di dalam API
SQLite	Database kecil untuk data **transaksi**: draf PO, hold, hitung fisik, idempotency key, circuit breaker, audit log, jejak langkah agent, antrian tugas agent.	File di Amazon EFS (online) atau laptop
Tiruan SAP	Meniru API Purchase Order SAP S/4HANA (OData V2 `API_PURCHASEORDER_PROCESS_SRV`, token CSRF, nomor PO 45xxxxxxxx). PO yang disetujui manajer dikirim ke sini.	Di dalam API
uvicorn	Server untuk menjalankan API di laptop (pengembangan).	Laptop

Guardrail di kode

Guardrail	Cara kerja
Filter 3σ	Qty PO di atas target stok + 3σ permintaan dianggap lonjakan tidak wajar (misalnya pesanan borongan) → ditahan.
Batas PO harian	Total nilai PO per toko per hari simulasi maksimal Rp100 juta (memajukan tanggal simulasi membuka anggaran hari berikutnya).
Circuit breaker	Kalau 3 PO ditahan guardrail dalam 60 menit, semua PO baru ditahan sampai manajer mereset.
Idempotency key	Request yang diulang dengan kunci yang sama tidak membuat PO atau hold ganda.
Draf secara default	Agent hanya menyusun draf; PO baru masuk SAP setelah manajer menekan 1-Click Approval. Satu draf per SKU.
Data Confidence	Kalau data stok tidak bisa dipercaya (di bawah 0,8, misalnya ada selisih hitung fisik), PO ditahan sampai stok diaudit.
Syarat Hold Reorder	Reorder hanya boleh ditahan kalau penjualan turun dan jenis produknya tidak komplementer; 10% tetap dibeli sebagai stok uji (exploration budget).

3. Agent ("otaknya")

Komponen	Untuk apa	Berjalan di
Strands Agents	SDK open-source dari AWS untuk membuat agent: mengatur putaran LLM berpikir → memanggil tool → membaca hasil → berpikir lagi sampai tugas selesai.	Bedrock AgentCore Runtime

Komponen	Untuk apa	Berjalan di
Amazon Bedrock AgentCore Runtime	Layanan AWS untuk menjalankan agent tanpa mengurus server. Setiap klik "Jalankan Agent" membuka satu sesi yang mengambil tugas dari antrian API, menjalankan agent, lalu ditutup 2 menit setelah selesai. Kodenya sama dengan agent di laptop (Python 3.13, ARM64, diunggah sebagai zip ke S3).	AWS (runtime `rexa_agent_toko`)
Claude Sonnet 4.6	LLM (model bahasa) agent: memilih tindakan dan menulis ringkasan; tidak menghitung angka (semua angka dari API). Satu run ±20 detik, ±27 detik dari klik sampai selesai.	Amazon Bedrock (Sydney)
Rexa Copilot	**Chatbot** per toko (Claude Haiku 4.5, 9 tool baca) dan **Agent Jaringan** untuk halaman Semua Toko (Claude Sonnet 4.6: menetapkan 3–5 toko prioritas dan transfer stok antar toko). Keduanya hanya membaca data. Copilot juga yang membangunkan agent toko di AgentCore.	AWS Lambda (di luar VPC)
Worker + Ollama (cadangan)	Worker di laptop (`python -m agent.worker`) mengambil tugas dari antrian yang sama kalau AgentCore bermasalah. Ollama + Qwen3 14B adalah LLM lokal kalau Bedrock tidak bisa dipakai. Tidak dipakai saat kondisi normal.	Laptop (opsional)
httpx	Pustaka yang dipakai agent untuk memanggil API lewat internet.	Di dalam agent

6 tools yang boleh dipakai agent (semuanya lewat API, jadi tetap dijaga guardrail)

Tool	Fungsi
list_prioritas	Daftar SKU yang perlu tindakan di satu toko: naik (kandidat draf PO) dan turun (kandidat hold).
get_sku_metrics	Detail satu SKU: stok, Vs, reorder point, saran PO, skor risiko, Data Confidence.
check_substitution	Cek apakah jenis produk punya peran komplementer (dari data keranjang Instacart).
hold_reorder	Tahan reorder SKU yang penjualannya turun dan tidak komplementer; menghitung modal yang diselamatkan.
suggest_reallocation	Saran pengalihan modal dari reorder yang ditahan ke SKU yang penjualannya naik.
draft_purchase_order	Susun draf PO; hasilnya DRAFT (menunggu manajer) atau DITAHAN_GUARDRAIL.

4. Dashboard dan Virtual HQ ("yang dilihat manajer")

Komponen	Untuk apa
React + Vite + Tailwind	Dashboard: panel agent (Plan → Reason → Act → Review), kartu PO menunggu persetujuan, tombol 1-Click Approval, Health Score, realokasi modal, halaman **Semua Toko** (statistik 80 toko dan transfer stok), **Rekap Agent** per hari simulasi (unduh CSV/PDF), kontrol tanggal simulasi, dan chatbot.
Virtual HQ (Phaser 4 + React 19)	Kantor 2D yang membaca API yang sama: staf membacakan laporan toko, Smart TV menampilkan statistik jaringan, Rexa menjalankan agent, dan mode Semua Toko untuk seluruh jaringan. Bisa dimainkan di HP (joystick dan tombol TAP). Dibuka dari tombol **Mode Virtual HQ** di dashboard.
Kiro	AI coding editor dari AWS, dipakai tim untuk melanjutkan dashboard (memberi poin tambahan penilaian).

5. AWS ("tempat semuanya online")

Layanan	Untuk apa
AWS Lambda (API)	Menjalankan API tanpa mengurus server; hanya aktif saat ada request sehingga murah. Function URL adalah alamat publiknya. Memori 1024 MB, container image Python 3.12, di dalam VPC.
AWS Lambda (Copilot)	Lambda kedua di **luar VPC**: Lambda API di VPC tidak punya akses internet keluar, jadi tidak bisa memanggil Bedrock atau AgentCore. Zip Python 3.12, ARM64.
Amazon Bedrock	Model Claude: Sonnet 4.6 untuk agent toko dan Agent Jaringan, Haiku 4.5 untuk chatbot (lebih hemat). Diakses lewat IAM role, tanpa API key.
Amazon Bedrock AgentCore Runtime	Tempat agent toko berjalan (lihat bagian 3). Ditagih per detik selama sesi hidup, jadi tidak ada biaya saat tidak dipakai.
Amazon S3	Menyimpan zip kode agent untuk AgentCore (±30 MB per deploy).
IAM	Izin per bagian: role AgentCore hanya boleh memanggil model Bedrock, membaca kodenya di S3, dan menulis log; Copilot hanya boleh membangunkan runtime agent itu.
Mangum	Adapter (penerjemah) supaya aplikasi FastAPI bisa berjalan di Lambda.
Docker + Amazon ECR	Docker membungkus API dan data DuckDB menjadi satu image; ECR menyimpan image itu di AWS.
Amazon EFS	"Hard disk bersama" untuk file SQLite. Tanpa EFS, data PO bisa hilang saat Lambda restart atau terbelah antar-salinan Lambda.
VPC + Security Group	Jaringan privat dan "satpam": Lambda API boleh mengakses EFS, sedangkan EFS tertutup untuk pihak lain.
AWS Amplify	Hosting dashboard dan Virtual HQ (`/hq/`): https://main.d30b19xkk8lknt.amplifyapp.com
CloudWatch Logs	Catatan aktivitas Lambda dan AgentCore untuk mencari error dan mengukur kecepatan.
AWS CLI + skrip PowerShell	Alat deploy: `deploy_lambda.ps1` (API), `deploy_copilot.ps1` (Copilot), `deploy_agent.ps1` (agent ke AgentCore), dan `deploy_dashboard.ps1` (dashboard + Virtual HQ).

6. Alur satu klik (yang ditunjukkan saat demo)

1. Manajer klik "Jalankan Agent" di dashboard atau di Virtual HQ (Amplify)

↓

2. API di Lambda mencatat tugas ke antrian: status MENUNGGU (SQLite di EFS)

↓

3. Dashboard meminta Lambda Copilot membangunkan agent di Bedrock AgentCore (InvokeAgentRuntime)

↓

4. Agent di AgentCore mengambil tugas: status BERJALAN; Claude Sonnet 4.6 mulai berpikir

↓

5. Agent memanggil tools di API: Plan → Reason → Act (prioritas, cek substitusi, hold, draf PO)

↓

6. Guardrail di API memeriksa setiap aksi; yang melanggar ditahan (DITAHAN_GUARDRAIL)

↓

7. Setiap langkah tampil langsung di dashboard; ringkasan agent = Review (±27 detik dari klik)

↓

8. Manajer klik "1-Click Approval" → API mengirim PO ke tiruan SAP → nomor PO 45xxxxxxxx

Langkah 4–7 adalah loop agentic yang diminta juri (plan, reason, act/tool, review). Langkah 8 adalah human-in-the-loop: tanpa persetujuan manajer, tidak ada PO yang masuk SAP.

7. Dari laptop ke AWS

Rencana awal memakai Amazon Bedrock. Pada 22 Oktober, kuota token Bedrock untuk model Amazon Nova di akun tim bernilai 0, jadi LLM sementara berjalan di laptop (Ollama) lewat worker yang mengambil tugas dari antrian API. Pada 25 Oktober Claude di Bedrock bisa dipakai, dan LLM pindah ke **Claude Sonnet 4.6**. Pada 26 Oktober agent sendiri dipindah ke **Amazon Bedrock AgentCore Runtime**, sehingga laptop tidak perlu menyala saat demo.

Kode agent sama di ketiga tahap; yang berubah hanya tempat menjalankannya, jadi dashboard dan API tidak perlu diubah. Agent dibangunkan lewat Lambda Copilot karena Lambda API di VPC tidak punya akses internet keluar dan browser tidak punya kredensial AWS. Worker di laptop tetap bisa dinyalakan sebagai cadangan.

8. Perintah penting

Perintah	Fungsi
`python -m engine.pipeline`	Membangun ulang data (Kaggle → DuckDB) di laptop
`python -m agent.reset_demo`	Mengosongkan data demo online (PO, hold, jejak agent; tanggal kembali ke 31 Des 2016)
`python -m agent.worker`	Worker cadangan di laptop (hanya kalau AgentCore bermasalah)
`deploy\deploy_lambda.ps1`	Deploy ulang API ke Lambda (butuh Docker Desktop)
`deploy\deploy_copilot.ps1`	Deploy ulang Rexa Copilot ke Lambda
`deploy\deploy_agent.ps1`	Deploy ulang agent ke Bedrock AgentCore
`deploy\deploy_dashboard.ps1`	Deploy ulang dashboard dan Virtual HQ ke Amplify